

UNIVERSIDAD TÉCNICA ESTATAL DE QUEVEDO

CARRERA:

INGENIERÍA EN SOFTWARE

INTEGRANTES:

LOZANO MORALES JOSE ALEJANDRO
SÁNCHEZ PILALOA ANDY PAUL
URBINA ROMERO ISAIAS ABRAHAM

CURSO:

7MO SOFTWARE "A"

DOCENTE:

ING. GUERRERO ULLOA GLEISTON CICERON

ASIGNATURA:

APLICACIONES DISTRIBUIDAS

TEMA:

PRÁCTICA EXPERIMENTAL:
COMUNICACIÓN ENTRE PROCESOS DISTRIBUIDOS CON SOCKETS TCP Y
gRPC

REPOSITORIO DEL PROYECTO:

[**ACCEDER AL REPOSITORIO EN GITHUB**](#)

PERIODO:

SÉPTIMO SEMESTRE 2026-2027
PPA

Índice

1. Introducción	3
2. Objetivos	3
2.1. Objetivo General	3
2.2. Objetivos Específicos	3
3. Marco Teórico	3
3.1. Sistemas Distribuidos	3
3.2. Sockets TCP	4
3.3. RPC y gRPC	4
3.4. Protocol Buffers	4
3.5. Relojes Lógicos de Lamport	4
4. Metodología	4
5. Implementación	5
5.1. Implementación TCP	5
5.2. Implementación gRPC	6
5.3. Ejecución y pruebas en IntelliJ IDEA	6
6. Resultados	7
7. Análisis Comparativo	9
8. Conclusiones	9

Resumen

Los sistemas distribuidos modernos requieren mecanismos de comunicación eficientes capaces de mantener sincronización, tolerancia a fallos y escalabilidad. En esta práctica experimental se implementó un sistema distribuido utilizando Python, Sockets TCP y gRPC con Protocol Buffers, integrando relojes lógicos de Lamport para mantener el orden de eventos entre múltiples nodos distribuidos.

La práctica consistió en desarrollar un servidor y tres nodos clientes ejecutados localmente, permitiendo el intercambio de mensajes mediante dos mecanismos de comunicación: Sockets TCP y gRPC. Además, se realizaron pruebas de latencia utilizando `time.perf_counter()` y almacenamiento de resultados en archivos CSV para su posterior análisis comparativo.

Finalmente, se evaluó el comportamiento de ambos mecanismos de comunicación considerando latencia, rendimiento y facilidad de implementación dentro de un entorno distribuido.

1. Introducción

Los sistemas distribuidos permiten la cooperación entre múltiples nodos independientes mediante intercambio de mensajes y procesamiento paralelo [1]. Actualmente, este tipo de arquitecturas son ampliamente utilizadas debido a su capacidad para soportar alta concurrencia, escalabilidad y tolerancia a fallos.

Las arquitecturas distribuidas modernas suelen apoyarse en modelos de microservicios y comunicación remota entre procesos, permitiendo que distintos servicios funcionen de forma desacoplada y paralela [2]. Dentro de estos mecanismos de comunicación destacan tecnologías como Sockets TCP, RPC y gRPC, las cuales permiten la transmisión de datos entre nodos distribuidos.

TCP continúa siendo uno de los protocolos de transporte más utilizados debido a su confiabilidad y control de errores [3]. Por otra parte, gRPC se ha convertido en una alternativa moderna para la comunicación distribuida gracias a su bajo consumo de recursos y utilización de Protocol Buffers como mecanismo de serialización eficiente [4].

En sistemas distribuidos también resulta necesario mantener coherencia temporal entre eventos ejecutados en distintos nodos. Para ello, Lamport propuso los relojes lógicos, mecanismo que permite establecer un orden parcial entre eventos distribuidos mediante timestamps lógicos [5].

En esta práctica se implementó un sistema distribuido utilizando Python, incorporando comunicación mediante Sockets TCP y gRPC con Protocol Buffers, así como relojes lógicos de Lamport para sincronización de eventos y medición de latencia.

2. Objetivos

2.1. Objetivo General

Implementar y comparar mecanismos de comunicación entre procesos distribuidos utilizando Sockets TCP y gRPC con Protocol Buffers.

2.2. Objetivos Específicos

- Implementar comunicación distribuida utilizando Sockets TCP.
- Implementar comunicación distribuida utilizando gRPC y Protocol Buffers.
- Incorporar relojes lógicos de Lamport para la sincronización de eventos distribuidos.
- Medir latencia de transmisión utilizando `time.perf_counter()`.
- Comparar rendimiento y comportamiento entre TCP y gRPC.

3. Marco Teórico

3.1. Sistemas Distribuidos

Un sistema distribuido consiste en un conjunto de nodos independientes que cooperan mediante comunicación en red para funcionar como un sistema lógico unificado [1]. Estas arquitecturas permiten distribuir carga de trabajo, mejorar escalabilidad y aumentar tolerancia a fallos.

Los sistemas distribuidos modernos utilizan arquitecturas basadas en microservicios para manejar alta concurrencia y procesamiento paralelo [2]. Cada servicio puede ejecutarse de forma independiente y comunicarse mediante protocolos distribuidos.

3.2. Sockets TCP

TCP (Transmission Control Protocol) es un protocolo orientado a conexión que garantiza entrega confiable y ordenada de datos entre procesos distribuidos [6]. Este protocolo incluye mecanismos de control de errores, retransmisión y confirmación de recepción.

Diversos estudios comparativos demuestran que TCP mantiene alta confiabilidad y estabilidad en comunicaciones distribuidas, aunque puede presentar mayor latencia frente a protocolos más ligeros [3].

3.3. RPC y gRPC

RPC (Remote Procedure Call) permite ejecutar procedimientos remotos entre nodos distribuidos como si fueran llamadas locales [7]. Este paradigma simplifica el desarrollo de aplicaciones distribuidas al abstraer detalles de red y comunicación.

gRPC representa una evolución moderna del modelo RPC, desarrollado por Google, que utiliza HTTP/2 y Protocol Buffers para optimizar rendimiento, throughput y latencia [8]. Además, gRPC soporta comunicación asíncrona, streaming y serialización eficiente.

3.4. Protocol Buffers

Protocol Buffers es un mecanismo de serialización de datos desarrollado por Google para intercambio eficiente de información estructurada [4]. A diferencia de JSON, Protocol Buffers utiliza representación binaria compacta, reduciendo tamaño de mensajes y tiempo de transmisión.

El uso combinado de gRPC y Protocol Buffers mejora el rendimiento de aplicaciones distribuidas al reducir consumo de ancho de banda y latencia [4].

3.5. Relojes Lógicos de Lamport

Los relojes lógicos de Lamport permiten establecer orden temporal entre eventos distribuidos mediante timestamps lógicos [5]. Cada nodo mantiene un contador lógico local que se incrementa durante eventos internos y recepción de mensajes.

Lamport introdujo además la relación “happened before”, la cual permite modelar dependencias causales entre eventos distribuidos [5]. Posteriormente surgieron mejoras como Vector Clocks y modelos basados en grafos para optimizar representación de causalidad [9].

4. Metodología

La práctica fue desarrollada utilizando Python 3 como lenguaje principal de programación.

Se implementaron dos mecanismos de comunicación:

- Comunicación distribuida mediante Sockets TCP.

- Comunicación distribuida mediante gRPC y Protocol Buffers.

El sistema estuvo compuesto por:

- Un servidor TCP.
- Un servidor gRPC.
- Tres nodos clientes ejecutados localmente.

Los nodos distribuidos fueron ejecutados sobre localhost utilizando los puertos 5000, 5001 y 5002 para simular la comunicación distribuida entre procesos.

Cada nodo realizó intercambio de mensajes utilizando timestamps lógicos de Lamport para mantener orden temporal de eventos.

La sincronización lógica se implementó mediante:

$$C_i = \text{máx}(C_i, T_m) + 1$$

donde:

- C_i representa el reloj lógico local.
- T_m representa el timestamp recibido.

Cada nodo realizó 20 intercambios de mensajes utilizando relojes lógicos de Lamport para mantener sincronización temporal entre eventos distribuidos.

Posteriormente se realizaron 100 envíos adicionales para medir latencia promedio utilizando:

```
time.perf_counter()
```

Los resultados fueron almacenados en archivos CSV para análisis estadístico posterior.

Además, el desarrollo del proyecto fue gestionado mediante un repositorio Git compartido entre los integrantes del equipo. Cada integrante realizó commits significativos relacionados con implementación, configuración, análisis y documentación del sistema distribuido.

5. Implementación

5.1. Implementación TCP

La implementación TCP consistió en un servidor capaz de recibir mensajes JSON desde múltiples nodos clientes.

Cada mensaje contenía:

- Nombre del nodo emisor.
- Timestamp lógico.
- Contenido del mensaje.

El servidor procesaba los mensajes y respondía con confirmaciones utilizando timestamps actualizados mediante el algoritmo de Lamport.

5.2. Implementación gRPC

La implementación gRPC utilizó archivos `.proto` para definir estructuras de mensajes y servicios remotos.

Posteriormente se generaron automáticamente los archivos:

- `messaging_pb2.py`
- `messaging_pb2_grpc.py`

El sistema permitió comunicación eficiente entre cliente y servidor utilizando serialización binaria mediante Protocol Buffers.

5.3. Ejecución y pruebas en IntelliJ IDEA

Las pruebas experimentales fueron ejecutadas utilizando el entorno de desarrollo IntelliJ IDEA, permitiendo simular múltiples nodos distribuidos mediante diferentes instancias de ejecución local.

Durante la implementación TCP se ejecutó un servidor encargado de recibir mensajes JSON enviados por los clientes distribuidos. Cada cliente incrementaba su reloj lógico de Lamport antes de enviar mensajes y actualizaba el timestamp tras recibir respuestas del servidor.

```
Servidor TCP escuchando en localhost:5000
Conexión desde ('127.0.0.1', 49812)
Recibido de Nodo1 | Lamport: 1 | Mensaje: Mensaje 1 desde Nodo1
Recibido de Nodo1 | Lamport: 4 | Mensaje: Mensaje 2 desde Nodo1
Recibido de Nodo1 | Lamport: 7 | Mensaje: Mensaje 3 desde Nodo1
Recibido de Nodo1 | Lamport: 10 | Mensaje: Mensaje 4 desde Nodo1
Recibido de Nodo1 | Lamport: 13 | Mensaje: Mensaje 5 desde Nodo1
Recibido de Nodo1 | Lamport: 16 | Mensaje: Mensaje 6 desde Nodo1
Recibido de Nodo1 | Lamport: 19 | Mensaje: Mensaje 7 desde Nodo1
```

Figura 1: Ejecución del servidor TCP en IntelliJ IDEA

La figura anterior muestra la ejecución del servidor TCP encargado de procesar mensajes provenientes de múltiples nodos distribuidos.

```
Enviado: Mensaje 94 desde Nodo1 | Respuesta: ACK recibido: Mensaje 94 desde Nodo1 | Lamport actual: 342 | Latencia: 0.0790 ms
Enviado: Mensaje 95 desde Nodo1 | Respuesta: ACK recibido: Mensaje 95 desde Nodo1 | Lamport actual: 345 | Latencia: 0.0788 ms
Enviado: Mensaje 96 desde Nodo1 | Respuesta: ACK recibido: Mensaje 96 desde Nodo1 | Lamport actual: 348 | Latencia: 0.0954 ms
Enviado: Mensaje 97 desde Nodo1 | Respuesta: ACK recibido: Mensaje 97 desde Nodo1 | Lamport actual: 351 | Latencia: 0.0880 ms
Enviado: Mensaje 98 desde Nodo1 | Respuesta: ACK recibido: Mensaje 98 desde Nodo1 | Lamport actual: 354 | Latencia: 0.0725 ms
Enviado: Mensaje 99 desde Nodo1 | Respuesta: ACK recibido: Mensaje 99 desde Nodo1 | Lamport actual: 357 | Latencia: 0.0738 ms
Enviado: Mensaje 100 desde Nodo1 | Respuesta: ACK recibido: Mensaje 100 desde Nodo1 | Lamport actual: 360 | Latencia: 0.0754 ms
Latencia promedio TCP: 0.0663 ms
Archivo generado: data/latency_sockets.csv
```

Figura 2: Comunicación entre nodos TCP y actualización de Lamport

La figura anterior evidencia el intercambio de mensajes entre nodos TCP junto con la actualización de los timestamps lógicos de Lamport.

Posteriormente se realizaron pruebas utilizando gRPC y Protocol Buffers. En este caso, el sistema utilizó serialización binaria y servicios remotos definidos mediante archivos `.proto`.

```
Enviado: Mensaje gRPC 94 desde Nodo6RPC1 | Respuesta: ACK gRPC: Mensaje gRPC 94 desde Nodo6RPC1 | Lamport actual: 342 | Latencia: 0.4644 ms
Enviado: Mensaje gRPC 95 desde Nodo6RPC1 | Respuesta: ACK gRPC: Mensaje gRPC 95 desde Nodo6RPC1 | Lamport actual: 345 | Latencia: 0.5043 ms
Enviado: Mensaje gRPC 96 desde Nodo6RPC1 | Respuesta: ACK gRPC: Mensaje gRPC 96 desde Nodo6RPC1 | Lamport actual: 348 | Latencia: 0.4841 ms
Enviado: Mensaje gRPC 97 desde Nodo6RPC1 | Respuesta: ACK gRPC: Mensaje gRPC 97 desde Nodo6RPC1 | Lamport actual: 351 | Latencia: 0.4269 ms
Enviado: Mensaje gRPC 98 desde Nodo6RPC1 | Respuesta: ACK gRPC: Mensaje gRPC 98 desde Nodo6RPC1 | Lamport actual: 354 | Latencia: 0.3887 ms
Enviado: Mensaje gRPC 99 desde Nodo6RPC1 | Respuesta: ACK gRPC: Mensaje gRPC 99 desde Nodo6RPC1 | Lamport actual: 357 | Latencia: 0.3860 ms
Enviado: Mensaje gRPC 100 desde Nodo6RPC1 | Respuesta: ACK gRPC: Mensaje gRPC 100 desde Nodo6RPC1 | Lamport actual: 360 | Latencia: 0.3749 ms
Latencia promedio gRPC: 0.4110 ms
Archivo generado: data/latency_grpc.csv
```

Figura 3: Comunicación distribuida utilizando gRPC

La figura anterior muestra la ejecución de clientes gRPC realizando intercambio de mensajes y recepción de confirmaciones desde el servidor distribuido.

Finalmente, se ejecutó el archivo `analisis.py`, encargado de procesar los archivos CSV generados durante las pruebas experimentales para calcular estadísticas y generar gráficas comparativas.

```
C:\Users\franc\OneDrive\Escritorio\Aplicaciones-Distribuidas\PracticaPython\.venv\Scripts\python.exe C:\Users\franc\OneDrive\Escritorio\Aplicaciones-Distribuidas_7moSemestre\OA_SUM_02\analisis.py

Resumen estadístico:
  tecnologia  media  mediana  desviacion_estandar  percentil_95
0  Sockets TCP  0.090681  0.08420      0.034172      0.142305
1      gRPC    0.514380  0.46865      0.173109      0.781745

Archivos generados:
data/resumen_estadistico.csv
docs/figures/boxplot_latencias.png
docs/figures/latencia_promedio.png
```

Figura 4: Ejecución del análisis estadístico en Python

La figura anterior presenta la ejecución del análisis de latencias utilizando Python y bibliotecas estadísticas como Pandas y Matplotlib.

6. Resultados

Tras ejecutar las pruebas de comunicación distribuida mediante Sockets TCP y gRPC, se obtuvieron los resultados de latencia mostrados a continuación.

Tecnología	Media (ms)	Desviación Estándar
Sockets TCP	0.090	0.034
gRPC	0.514	0.173

Cuadro 1: Resultados estadísticos de latencia

Los resultados muestran que Sockets TCP presentó una menor latencia promedio respecto a gRPC durante las pruebas realizadas localmente.

Asimismo, gRPC presentó mayor variabilidad debido a las capas adicionales de serialización, manejo de HTTP/2 y abstracción propia del framework.

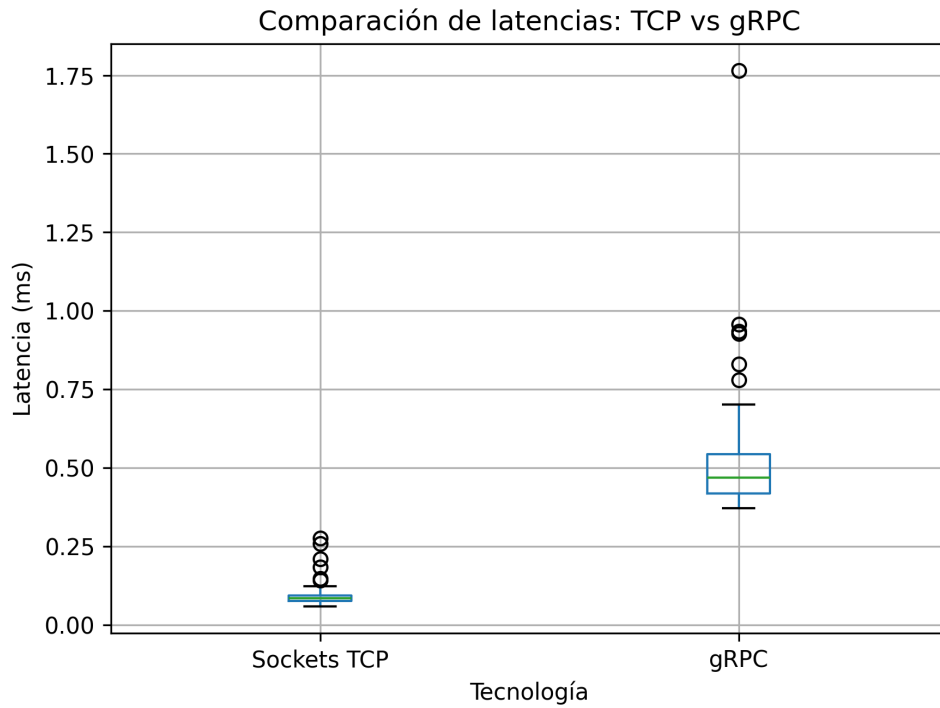


Figura 5: Comparación de latencias entre TCP y gRPC

La figura anterior muestra la distribución estadística de las latencias registradas para ambas tecnologías. Se puede observar que TCP mantiene valores más estables y uniformes, mientras que gRPC presenta una dispersión ligeramente superior.

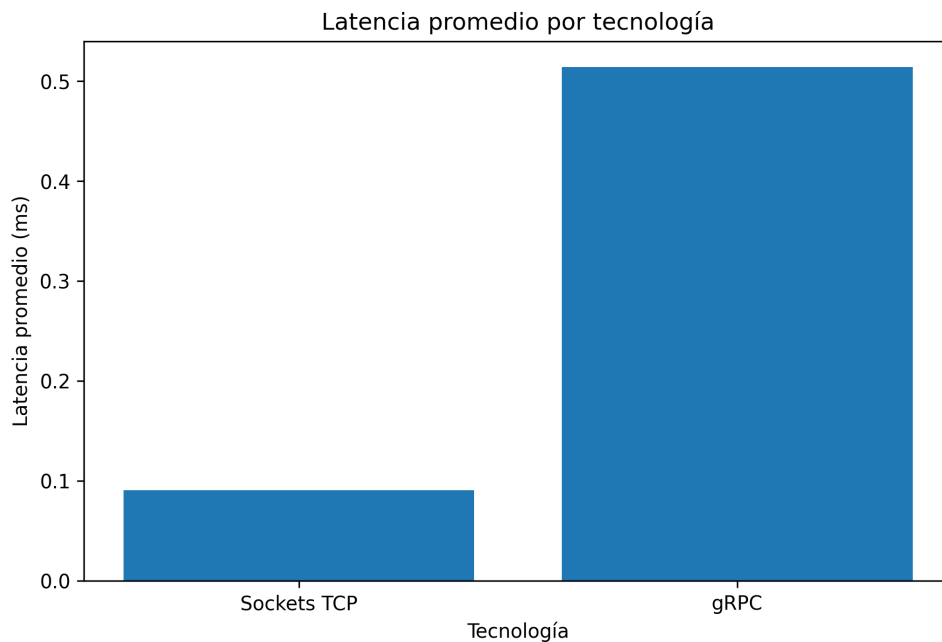


Figura 6: Latencia promedio de TCP y gRPC

La figura anterior representa la latencia promedio obtenida durante las pruebas experimentales realizadas sobre localhost utilizando tres nodos distribuidos.

7. Análisis Comparativo

Los resultados experimentales obtenidos permitieron comparar el comportamiento de Sockets TCP y gRPC dentro de un entorno distribuido local.

Sockets TCP presentó una latencia promedio de aproximadamente 0.090 ms, mostrando un comportamiento más rápido y estable durante la transmisión de mensajes. Esto ocurre debido a que TCP realiza comunicación directa entre procesos con una menor sobrecarga de procesamiento y serialización.

Por otra parte, gRPC presentó una latencia promedio de aproximadamente 0.514 ms. Aunque la latencia fue superior respecto a TCP, gRPC incorpora múltiples ventajas modernas relacionadas con escalabilidad, serialización eficiente y arquitectura basada en servicios.

El uso de HTTP/2 y Protocol Buffers introduce cierta sobrecarga adicional durante la transmisión, pero al mismo tiempo permite optimizar la comunicación entre microservicios y aplicaciones distribuidas modernas [8].

Asimismo, Protocol Buffers reduce considerablemente el tamaño de los mensajes transmitidos en comparación con formatos tradicionales como JSON [4], permitiendo una comunicación más eficiente en sistemas distribuidos de gran escala.

En consecuencia, TCP resulta adecuado para aplicaciones simples que requieren baja latencia y comunicación directa, mientras que gRPC representa una alternativa más robusta y escalable para arquitecturas modernas basadas en microservicios.

8. Conclusiones

La práctica permitió implementar y comparar dos mecanismos modernos de comunicación distribuida: Sockets TCP y gRPC con Protocol Buffers.

La implementación de relojes lógicos de Lamport permitió comprender la importancia de mantener coherencia temporal y orden de eventos dentro de sistemas distribuidos.

Los resultados experimentales demostraron que Sockets TCP presenta menor latencia debido a su comunicación directa y menor sobrecarga de procesamiento.

Por otra parte, gRPC demostró ventajas importantes relacionadas con escalabilidad, serialización eficiente, organización modular y soporte para arquitecturas modernas distribuidas.

Finalmente, el desarrollo de esta práctica permitió fortalecer conocimientos relacionados con sistemas distribuidos, sincronización lógica, protocolos de comunicación y análisis de rendimiento en entornos distribuidos.

Referencias

- [1] K. Guo and R. Gao, “Distributed healthcare information system based on high-concurrency microservice architecture,” in *2023 IEEE 4th International Conference on Information Science, Parallel and Distributed Systems (ISPDS)*, 2023.
- [2] C. Kröher, L. Gerling, and K. Schmid, “Combining distributed and central control for self-adaptive systems of systems,” in *2022 IEEE 42nd International Conference on Distributed Computing Systems Workshops (ICDCSW)*, 2022.
- [3] S. Jeddou, L. Diez, N. Abdellah, A. Baina, and R. A. Calvo, “On the performance of transport protocols over mmwave links: Empirical comparison of tcp and quic,” *IEEE Open Journal of the Communications Society*, 2023.
- [4] A. Sangwai, S. Sapale, S. Ghodake, and R. Jadhav, “Barricading system - system communication using grpc and protocol buffers,” in *2023 5th Biennial International Conference on Nascent Technologies in Engineering (ICNTE)*, 2023.
- [5] R. K. Ghosh and H. Ghosh, “Clock synchronization and event ordering,” in *Distributed Systems: Theory and Applications*. Wiley, 2023.
- [6] K. Karpov, D. Kachan, E. Siemens, and M. Popova, “A comparative study of reliable multideestination data transport protocol with tcp and udp in a point-to-multipoint streaming environment,” in *2023 IEEE International Black Sea Conference on Communications and Networking (BlackSeaCom)*, 2023.
- [7] J. Zhang, Z. Shu, and L. Luo, “An adaptive rpc mechanism for performance and node fault tolerance optimization in hdfs,” in *2022 IEEE 24th International Conference on High Performance Computing & Communications; 8th International Conference on Data Science & Systems; 20th International Conference on Smart City; 8th International Conference on Dependability in Sensor, Cloud & Big Data Systems & Application*, 2022.
- [8] L. Geng, H. Wang, J. Meng, D. Fan, S. Ben-Romdhane, and H. K. Pichumani, “Rr-compound: Rdma-fused grpc for low latency, high throughput, and easy interface,” *IEEE Transactions on Parallel and Distributed Systems*, 2024.
- [9] S. Sokoto and E. Onur, “Sglc: A logical clock using succinct graphs,” in *2022 30th Signal Processing and Communications Applications Conference (SIU)*, 2022.